CenseoAI

SCALE THE PROVEN

Build Your Own “AI Brain” Step-by-Step Guide

The $2,000 “AI Brain” install, built yourself with a Claude subscription and an afternoon — context, connections, skills, agents, and evals.

Build Your Own "AI Brain" — Step-by-Step Guide

A how-to guide for building the system sold as the **"AI Brain"** (Kieren Newborn's $99 webinar / $2,000 done-with-you install) — using nothing but the concepts already taught in this course: `CLAUDE.md` context files (Module 1), MCP tool connections (Module 2), sub-agents (Module 3), and skills with evals (Modules 1 & 4).

Total cost to build it yourself: **a Claude subscription (~$20/month) and an afternoon.**

What the product actually is

The "AI Brain" is marketed as a "complete pre-built AI Brain with 8 AI employees and 15 skills." Strip away the branding and it's a well-organized version of exactly what this course teaches:

Marketing term	What it really is	Course module
"Your business context loaded"	A `CLAUDE.md` + a folder of context files (who you are, what you sell, your tone, your clients)	Module 1
"Hands, not just a memory"	MCP connectors to Gmail, Google Calendar, and your other apps	Module 2
"8 AI employees"	8 sub-agent files in `.claude/agents/` — persona + instructions + tools	Module 3
"15 skills"	15 `SKILL.md` files in `.claude/skills/` — reusable slash-command workflows	Modules 1 & 2
"Local, private, and yours"	Plain markdown files on your own machine (an Obsidian vault works well)	—
"Second brain that remembers"	A `memory.md` decisions log Claude reads every session	Module 1 / this repo

The daily interface is the Claude desktop app or Claude Code; the knowledge layer is a folder of markdown notes (Obsidian is the popular choice because it's free, local, and reads the same markdown files Claude writes).

> **Honesty note:** the exact names of the product's 8 employees and 15 skills are behind the paywall. Everything below is reconstructed from the product's public claims plus the open-source "AI second brain" ecosystem that uses the identical architecture. Functionally, you end up with the same system — and yours is editable.

Prerequisites

- **Claude subscription** (~$20/month Pro)
- **Claude Code** installed (`npm install -g @anthropic-ai/claude-code`) — see the root README setup guide. The Claude desktop app also works for daily use; Claude Code is best for the build.
- **Obsidian** (free, obsidian.md) — optional but recommended. It's just a nice viewer/editor for the markdown vault you're about to create.
- **Node.js** installed (for MCP servers) — see Module 2.

No coding required. Every step is "create a text file" or "paste a prompt."

Step 1: Create the vault (the brain's filing cabinet)

Create one folder that will hold everything. If you use Obsidian, create it as a new vault.

```
my-ai-brain/
├── CLAUDE.md            ← master briefing (Step 2)
├── memory.md            ← decisions log (Step 3)
├── context/             ← who you are (Step 2)
├── inbox/               ← raw capture — braindumps, pasted notes
├── projects/            ← one note per active project
├── people/              ← one note per client/contact (your lightweight CRM)
├── daily/               ← morning briefings and daily notes
├── templates/           ← note templates
├── archive/             ← finished stuff
└── .claude/
    ├── skills/          ← the 15 skills (Step 5)
    └── agents/          ← the 8 AI employees (Step 6)
```

Ask Claude to do it for you — open Claude Code in the empty folder and paste:

```
Create this folder structure for my AI second brain: context/, inbox/,
projects/, people/, daily/, templates/, archive/, .claude/skills/,
.claude/agents/. Add a short README.md in each folder explaining what goes in it.
```

Why this works: Claude only knows what's in its context. A predictable folder structure means every skill and agent knows exactly where to read and write. This is the same reason this course repo has one folder per module.

Step 2: Load your business context (Module 1 practice)

This is the part the sales page calls "the context that makes it useful instead of generic." It's a `CLAUDE.md` plus a handful of context notes — Assignment 1a, applied to your business instead of a product.

2a. The master `CLAUDE.md`

Open Claude Code in the vault root and paste:

```
I'm building a personal AI Brain for my business. Help me create a CLAUDE.md
for this vault. Interview me for: who I am and my role, what my business does,
who my clients are, my tone of voice, and how I want you to respond. Then
write the file. Include a "Vault conventions" section explaining what lives
in each folder and a rule to check memory.md at the start of every session.
```

A good result covers the same four questions as Module 1: who you are, what the project (business) is, how Claude should respond, and your conventions. Use module-1/CLAUDE-template.md as a reference for the shape.

2b. The context folder

Create six short notes in `context/`. One prompt does it:

```
Interview me one file at a time and write these notes in context/:
- business.md  — what I sell, to whom, at what price, and how
- icp.md       — my ideal customer: pains, goals, objections
- brand.md     — my tone of voice with 3 real writing samples I'll paste
- strategy.md  — my goals for this quarter and the metrics that matter
- team.md      — who's on my team and what they own
- operator.md  — me: role, strengths, what I never want to do manually
```

Check it's working (same test as Module 1): start a fresh session and ask *"What do you know about my business based on what you've read?"* If the answer is generic, the context files are too thin — fill them in.

Best practice from this course: put real writing samples in `brand.md`. "Draft replies in your voice" — the product's flagship claim — is entirely a function of how good your voice samples are.

Step 3: Add persistent memory

Create `memory.md` in the vault root — same pattern as this repo's memory.md:

```
# Memory

Running log of decisions and context that isn't obvious from the notes.

## Decisions
| Decision | Reason | Date |
|----------|--------|------|

## Active commitments
- [ ]

## How to update this file
After any key decision, new client, or changed priority, add a row.
```

Then add one line to your `CLAUDE.md`: `Read memory.md at the start of every session. Update it when I make a decision or commit to something.`

That's the whole "brain that remembers" feature. (If you outgrow it, docs/resources.md covers heavier memory setups like Obsidian + Graphify — but as the course says: `CLAUDE.md` handles 90% of what you need.)

Step 4: Give it hands — connect your tools with MCP (Module 2 practice)

The product's "connects to email, calendar, and the apps you already run" is MCP — the same setup as Assignment 2b, pointed at Google instead of a browser.

> **Check what you already have first.** If you use the Claude desktop app or claude.ai, many connections are one click away as **Connectors** (Settings → Connectors): Gmail, Google Calendar, Google Drive, Outlook/Microsoft 365, Airtable, Canva, and more. And if you've been using Claude for a while, some may already be connected — ask Claude *"List the MCP tools available to you"* before installing anything. Only fall back to the terminal commands below for servers that aren't offered as Connectors. **On Outlook instead of Gmail?** Connect Microsoft 365 — every email/calendar skill in this guide works the same; only the tool names differ.

With Claude Code **not** running:

```
# Email + Calendar (Google Workspace MCP — pick one from the MCP registry)
claude mcp add google-workspace npx @your-chosen/google-workspace-mcp

# A real browser for research (exactly Assignment 2b)
npm install -g @playwright/mcp
claude mcp add playwright npx @playwright/mcp
```

Restart Claude Code and verify, exactly as in Module 2:

```
List the MCP tools available to you.
```

You should see Gmail/Calendar tools and the Playwright browser tools. If you use the Claude desktop app day-to-day, add the same servers as "Connectors" in its settings.

Safety rule — write it into your `CLAUDE.md`:

```
Never send an email, create a calendar event, or take any external action
without showing me the draft and getting my explicit OK first.
```

The product makes the same promise ("never sends without you"). With MCP it's one line of context, not a feature you pay for.

Make it enforced, not just promised. A `CLAUDE.md` rule is an instruction Claude follows; a permission rule is one the harness enforces. In Claude Code, run `/permissions` and set any send/create/delete tool (send email, create calendar event) to **ask** — then every outbound action requires your click, no matter what the prompt says. Belt and suspenders: keep both.

Troubleshooting: MCP servers load at startup only — fully restart Claude after adding one. If a server won't connect, check `~/.claude/settings.json` is valid JSON (Module 2 troubleshooting section).

Step 5: Install the 15 skills (Modules 1 & 2 practice)

Skills are the "jobs you hand over." Each is a folder in `.claude/skills/` containing a `SKILL.md` — the exact format from Assignment 1b and 2a: frontmatter with `name:` and a trigger-rich `description:`, then the instructions.

The 15 below reconstruct the product's confirmed capabilities (inbox triage, drafting in your voice, one-view client briefings, daily briefings) plus the standard second-brain set:

#	Skill	What it does
1	`/daily-brief`	Reads today's calendar + recent email + open tasks → one morning briefing saved to `daily/`
2	`/inbox-triage`	Reads the inbox via MCP, sorts into act-now / reply / read-later / ignore
3	`/email-draft`	Drafts a reply in your voice (uses `context/brand.md`) — never sends
4	`/client-brief`	Pulls everything about one client — `people/` note, recent emails, calendar history — into one view
5	`/meeting-prep`	Attendees, context, agenda, and 3 talking points before a meeting
6	`/meeting-debrief`	Paste notes → decisions, action items, updates to `people/` and `memory.md`
7	`/braindump`	Paste raw thoughts → classified and filed into the right folders
8	`/content-draft`	Blog/newsletter draft in your voice from a topic
9	`/social-post`	Platform-specific post from an idea or a long piece
10	`/proposal`	Client proposal from a short brief (problem, solution, price, next steps)
11	`/research`	Web research with sources (uses Playwright MCP)
12	`/follow-up`	Scans `people/` + sent mail for overdue follow-ups
13	`/weekly-review`	Synthesizes the week's daily notes against `context/strategy.md`
14	`/sop-writer`	Turns "how I do X" into a step-by-step process doc
15	`/capture`	Files a new learning/link/idea into `projects/` or `archive/` with backlinks

How to build them fast

Don't hand-write 15 files. Have Claude generate them, then edit — the same "read the SKILL.md, then customize it" loop from Assignment 1b:

```
Create a skill at .claude/skills/daily-brief/SKILL.md. Frontmatter: name
"daily-brief", description "Generate my morning briefing. Use when I ask for
my daily brief, morning summary, or what's on today." Instructions: read
today's events via the calendar MCP, scan unread email via the Gmail MCP,
check open items in memory.md and projects/, then write a briefing to
daily/YYYY-MM-DD.md with sections: Today's meetings (with one-line prep
notes), Needs a reply, Top 3 priorities, Flags. Keep it under 300 words.
```

Repeat for each row, or paste the whole table and ask Claude to generate all 15. Then **open each file and read it** — it's exactly what Claude will follow, and editing it is how you make the brain yours.

You already have working examples to crib from in this repo: prd-generator (interview-then-generate pattern), youtube-researcher (MCP-powered skill).

Ready-made starters

Nine of the highest-value skills are pre-built in docs/ai-brain-skills/ — copy any folder into your vault's `.claude/skills/` and edit the specifics (your CRM base, your inbox, your folders):

- **Daily operations:** inbox-triage, meeting-prep, follow-up, braindump
- **Client work:** client-brief, deal-prep, client-report, aeo-snapshot
- **Process:** sop-writer

Each one states which connections it uses and degrades gracefully when one is missing.

Step 6: Hire the 8 AI employees (Module 3 practice)

The "AI employees" are sub-agents — Assignment 3a-3e with business personas instead of PM ones. Each is one markdown file in `.claude/agents/` with frontmatter (name, trigger description, model, tools) and a job description. Claude delegates to them automatically when your request matches; each works in its own context window and hands back a clean summary.

The 8, with models chosen by the Module 3 rule (Opus for research, Sonnet for analysis, Haiku for high-volume copy):

#	Employee	Model	Tools	Job
1	`chief-of-staff`	Sonnet	Read	Triage, prioritization, daily/weekly planning
2	`email-manager`	Sonnet	Read + Gmail MCP	Inbox processing and reply drafting
3	`researcher`	Opus	WebSearch, WebFetch, Read	Market, client, and competitor research
4	`content-writer`	Sonnet	Read	Long-form content in your voice
5	`copywriter`	Haiku	Read	High-volume short copy: subjects, posts, CTAs
6	`sales-assistant`	Sonnet	Read	Client briefs, proposals, follow-up drafts
7	`ops-manager`	Sonnet	Read, Glob	SOPs, process docs, vault housekeeping
8	`strategist`	Opus	Read	Weekly reviews, goal tracking, "what should I focus on"

Example file — `.claude/agents/email-manager.md` (same shape as every agent in module-3):

```
---
name: email-manager
description: Process the inbox, categorize messages, and draft replies in
  the operator's voice. Use when asked to check email, triage the inbox,
  or draft a reply.
model: claude-sonnet-5
---

You are the operator's email manager.

Voice: match context/brand.md exactly — study the writing samples before
drafting anything.

For triage, return: **Act now** / **Needs a reply** (with one-line summary
each) / **Read later** / **Ignore**.

For drafts: write the reply in full, under 150 words unless the thread
demands more. Present it for approval. NEVER send anything yourself.
```

Build the rest the same way, or paste the table above and ask Claude to generate all 8, then edit the descriptions until delegation triggers reliably.

Two frontmatter notes: - **`tools:`** omitting the field (as above) lets the agent use every tool you've connected — the right default for an email manager that needs your email MCP. List specific tools only to *restrict* an agent (e.g. give `content-writer` just `Read` so it can never touch email). An agent whose tool list doesn't include the tools its job needs will silently fail at that job. - **`model:`** model names change over time — check the current list with `/model` before hardcoding, or omit the field entirely to inherit whatever model your session runs. Omitting is the low-maintenance default.

Module 3 best practices that make this work: - The `description` field is the trigger. "Use when asked to check email, triage the inbox, or draft a reply" delegates reliably; "handles email" doesn't. - **One owner per job.** If `/email-draft` (a skill) and `email-manager` (an agent) both exist, keep the actual drafting instructions in ONE place — the skill — and have the agent's file say "follow .claude/skills/email-draft/SKILL.md for drafts." Duplicate instructions drift apart and you'll get different drafts depending on which one fired. - **Pipelines need no plumbing** — you're the handoff point. One prompt runs a chain: *"Research this prospect, then draft an intro email in my voice, then give me a one-page brief for the call."* That's researcher → email-manager → sales-assistant, and it's the product's "hand a whole job to AI" demo. - Skills vs agents: skills you trigger with `/command` (workflows you control); agents trigger themselves when the request matches (specialists you delegate to).

Step 7: Test it like you'd ship it (Module 4 practice)

The difference between a demo and a system you trust is evals — Assignment 4a, applied to your brain.

1. Write a ground-truth table in `docs/eval-ground-truth.md`: 10 real inputs (an actual client email, a real meeting's notes, a real morning's calendar) and what a good output looks like for each.
2. Copy `/skill-evaluator` from module-4/.claude/skills/skill-evaluator into your vault and run it against your most-used skills (`/email-draft` and `/daily-brief` first — they're the ones you'll trust blindly soonest).
3. Score, fix the SKILL.md, rerun. Ship at 8/10 or better; below 5/10, rethink the skill's job definition.

The single highest-leverage eval: give `/email-draft` five real emails you already answered, and compare Claude's drafts to what you actually sent. Every gap is a missing line in `context/brand.md`.

Step 8: Run the daily loop

The product's pitch is a rhythm, not a feature. Yours:

When	What you type
Morning	`/daily-brief`
Then	`/inbox-triage` → approve/edit the drafts it queues
Before each meeting	`/meeting-prep for my 2pm`
After each meeting	`/meeting-debrief` + paste notes
Anytime	`/braindump` whatever's in your head
Friday	`/weekly-review`

Two weeks of this loop and the compounding starts: `people/` notes get richer, `memory.md` accumulates decisions, and every skill output gets sharper because the context it reads keeps improving. That flywheel — not any single feature — is what the $2,000 install is actually selling.

Step 9: Put the loop on a schedule

Step 8 as written is manual — you type `/daily-brief` every morning, forever. The upgrade that turns the brain from a tool you drive into a system that taps you on the shoulder is **scheduling**, and Claude supports it natively:

- **Claude Code on the web / desktop** supports scheduled tasks ("Routines"): a prompt that fires on a cron schedule into a session, with an optional push or email notification when it finishes. Set three:

Schedule	Prompt it fires
Weekdays 7:00am	"Run /daily-brief and notify me"
Monday 8:00am	"Run /follow-up and flag anyone overdue"
Friday 4:00pm	"Run /weekly-review"

- **In a terminal-only setup**, the same thing is one `cron` entry running `claude -p "/daily-brief"` — ask Claude to write it for you.

Your approval rules still hold: a scheduled run can *prepare* drafts and briefings, but the permission settings from Step 4 mean nothing goes out without you. You wake up to a finished briefing and a queue of drafts to approve — which is precisely the daily experience the paid product demos.

Upgrade paths: if you already run a business stack

The steps above assume you're starting from markdown files and Google. If you already run real tools, wire the brain into them instead of duplicating them:

Instead of…	If you have…	Do this
`people/` markdown notes as your CRM	**Airtable** (or any CRM with an MCP/Connector)	Keep structured client data (status, value, last contact) in the CRM; keep narrative notes in `people/`. Point `/client-brief` and `/follow-up` at both.
Pasting meeting notes into `/meeting-debrief`	**Granola**, Fireflies, or any transcript tool with a connector	Have the skill pull the transcript itself — debriefs happen without you copying anything.
Gmail-only email skills	**Outlook / Microsoft 365**	Connect the Microsoft 365 Connector; `/inbox-triage` should sweep every inbox you actually use.
Generic `/research` for client work	**Ahrefs**, Semrush, or your industry's data tool	Build skills on your data moat — see client-report and aeo-snapshot for the pattern.
Text-only content skills	**Canva** or a design-tool connector	Let `/social-post` and `/client-report` hand finished copy to a branded template instead of stopping at text.

The principle: the brain's job is to *join* your tools, not replace them. Every connector you already have is a skill input you don't have to build.

Version it (course convention)

Your brain is plain text, so treat it like this repo: make the vault a git repo, commit after every working step, and tag milestones (`v1.0` = context loaded, `v2.0` = skills live, `v3.0` = agents + MCP). `git checkout v1.0` restores any earlier state. Add `.claude/settings.local.json` and anything sensitive to `.gitignore` — and think twice before pushing a vault containing client emails to any remote at all; private repo or no remote.

What you built vs. what's being sold

	The $99 webinar / $2,000 install	This guide
Business context	Loaded on a call	Steps 2–3 (about an hour of interviews)
Tool connections	Configured for you	Step 4 (two terminal commands)
8 AI employees	Pre-built, fixed	Step 6 — yours, editable, model-per-job
15 skills	Pre-built, fixed	Step 5 — yours, editable, eval-tested
Quality assurance	"Refund if it doesn't work"	Step 7 — actual evals with your real data
Ongoing cost	Claude ~$20/mo	Claude ~$20/mo

The paid versions buy speed and hand-holding, which has real value for a non-technical buyer. But there is no proprietary technology in the product — it's `CLAUDE.md` + context files + skills + sub-agents + MCP, which is to say: Modules 1 through 4 of this course, pointed at your business instead of your product.

Troubleshooting

Claude feels generic despite the context files — The files are probably thin. Test: *"What do you know about my business?"* Whatever it can't answer, add.

Agents aren't triggering — Description too vague, or the file isn't at exactly `.claude/agents/<name>.md`. See Module 3 troubleshooting.

Skills aren't triggering — `.claude/` must be in the folder where you launched Claude. See Module 2 troubleshooting.

MCP tools missing — Full restart required after adding a server. Verify with *"List the MCP tools available to you."*

Email drafts don't sound like you — Add 2–3 more real samples to `context/brand.md`, including one where you say no to something. Tone lives in the edge cases.

Sources

Product claims: kierennewborn.com · the install offer · the AI Brain webinar page. Architecture cross-referenced against the open-source ecosystem using the same pattern: noahvnct's second-brain guide, AgriciDaniel/claude-obsidian, huytieu/COG-second-brain, eugeniughelbur/obsidian-second-brain, coleam00/second-brain-starter. The product's exact employee/skill names are paywalled; this guide reconstructs them from public claims and ecosystem norms.

Built with the practices from Claude Code in Practice — Modules 1–4.